

PROGRAMMING IN C

Course Code: **23CS101** • Semester: **1-1** • Department of Computer Science & Engineering

Course Scope & Objectives: Comprehensive foundation in procedural programming, algorithmic problem solving, memory manipulation, and low-level data representations.

Unit 1: Introduction to Algorithms, C Basics, and Control Structures

Key Syllabus Topics: Algorithms, Flowcharts, Structure of C Program, Data Types, Operators, Type Casting, Conditional Statements (if-else, switch-case), Iterative Loops (for, while, do-while), break/continue statements.

Theoretical Framework & Concepts:

Procedural programming principles in C emphasize structured decomposition. Variables define storage locations with specific bit allocations. Type conversions can be implicit (coercion) or explicit. Control flow is evaluated using boolean truthiness (non-zero is true). Short-circuit logic optimizes logical operations in condition checks.

Implementation / Key Formula Reference:

```
#include <stdio.h>
int main() {
    int n = 10, sum = 0;
    for(int i = 1; i <= n; i++) {
        sum += i;
    }
    printf("Sum of first %d natural numbers = %d\n", n, sum);
    return 0;
}
```

Frequently Asked University Exam Questions:

• **Q1: Explain the fundamental compilation phases of a C program.**

Solution: A: Preprocessing (macro expansion & header inclusion), Compilation (C code to assembly), Assembly (assembly to machine object code), and Linking (resolving external references & library linkage).

• **Q2: Differentiate between while and do-while loops with syntax.**

Solution: A: 'while' is an entry-controlled loop evaluating condition before execution; 'do-while' is exit-controlled, ensuring the body executes at least once before evaluating the condition.

Unit 2: Functions, Scope, and Storage Classes

Key Syllabus Topics: Function Prototypes, Call by Value vs Call by Reference, Recursion, Base Conditions, Variable Scope (Local, Global), Storage Classes (auto, register, static, extern).

Theoretical Framework & Concepts:

Functions provide modularity and code reuse. Recursive functions require a well-defined base case to avoid stack overflow. Storage classes dictate the scope, lifetime, and default initial value of variables. 'static' variables retain

values between successive function calls.

Implementation / Key Formula Reference:

```
int factorial(int n) {  
    if (n <= 1) return 1; // Base case  
    return n * factorial(n - 1); // Recursive step  
}
```

Frequently Asked University Exam Questions:

- **Q1: What are storage classes in C? Compare static and extern.**

Solution: A: Storage classes define scope and lifetime. 'static' keeps a variable alive throughout execution within file/function scope. 'extern' references a global variable defined across compilation units.

Unit 3: Arrays, Strings, and Pointer Fundamentals

Key Syllabus Topics: 1D and 2D Arrays, Matrix Operations, Character Arrays, String Manipulation (string.h functions), Pointer Arithmetic, Pointers to Arrays, Pointer to Functions.

Theoretical Framework & Concepts:

Arrays represent contiguous memory blocks. Pointers hold memory addresses; pointer arithmetic operates in multiples of the base data type's byte size. Dynamic string handling relies on null-terminators ('\0').

Implementation / Key Formula Reference:

```
void swap(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

Frequently Asked University Exam Questions:

- **Q1: Explain pointer arithmetic and memory addressing in C.**

Solution: A: Adding an integer k to a pointer (ptr + k) increments the memory address by k * sizeof(*ptr) bytes, ensuring precise indexing in array structures.

Unit 4: Structures, Unions, and Dynamic Memory Allocation

Key Syllabus Topics: struct Declaration, Access Operators, Nested Structures, Array of Structures, union vs struct, Bitfields, DMA functions: malloc(), calloc(), realloc(), free(), Memory Leaks & Dangling Pointers.

Theoretical Framework & Concepts:

Structures group heterogeneous data elements with distinct memory addresses. Unions share a single memory space equal to the largest member. Dynamic memory allocation operates on the heap via malloc/calloc; unreferenced allocations without free() cause memory leaks.

Implementation / Key Formula Reference:

```
struct Student {
    int roll;
    char name[50];
    float gpa;
};
struct Student *s = (struct Student*)malloc(sizeof(struct Student));
```

Frequently Asked University Exam Questions:

• **Q1: Differentiate between malloc() and calloc().**

Solution: A: malloc(size) allocates uninitialized raw bytes; calloc(n, size) allocates memory for n elements and initializes all bytes to zero.

Unit 5: File Handling and Preprocessor Directives

Key Syllabus Topics: Streams (stdin, stdout, stderr), File Open Modes (r, w, a, r+, rb, wb), Text vs Binary Files, Formatted I/O (fprintf, fscanf), Block I/O (fread, fwrite), Preprocessor Directives (#define, #include, #ifdef), Command Line Arguments (argc, argv).

Theoretical Framework & Concepts:

File operations interact with disk buffers via FILE pointers. Binary modes ('rb', 'wb') bypass OS character translation. Command line arguments enable runtime parameterization from shell prompts.

Implementation / Key Formula Reference:

```
FILE *fp = fopen("data.txt", "w");
if(fp != NULL) {
    fprintf(fp, "PBR VITS CSE Department\n");
    fclose(fp);
}
```

Frequently Asked University Exam Questions:

• **Q1: What are command line arguments in C? Explain argc and argv.**

Solution: A: 'argc' contains the argument count including executable name. 'argv' is an array of string pointers pointing to each argument passed at program launch.